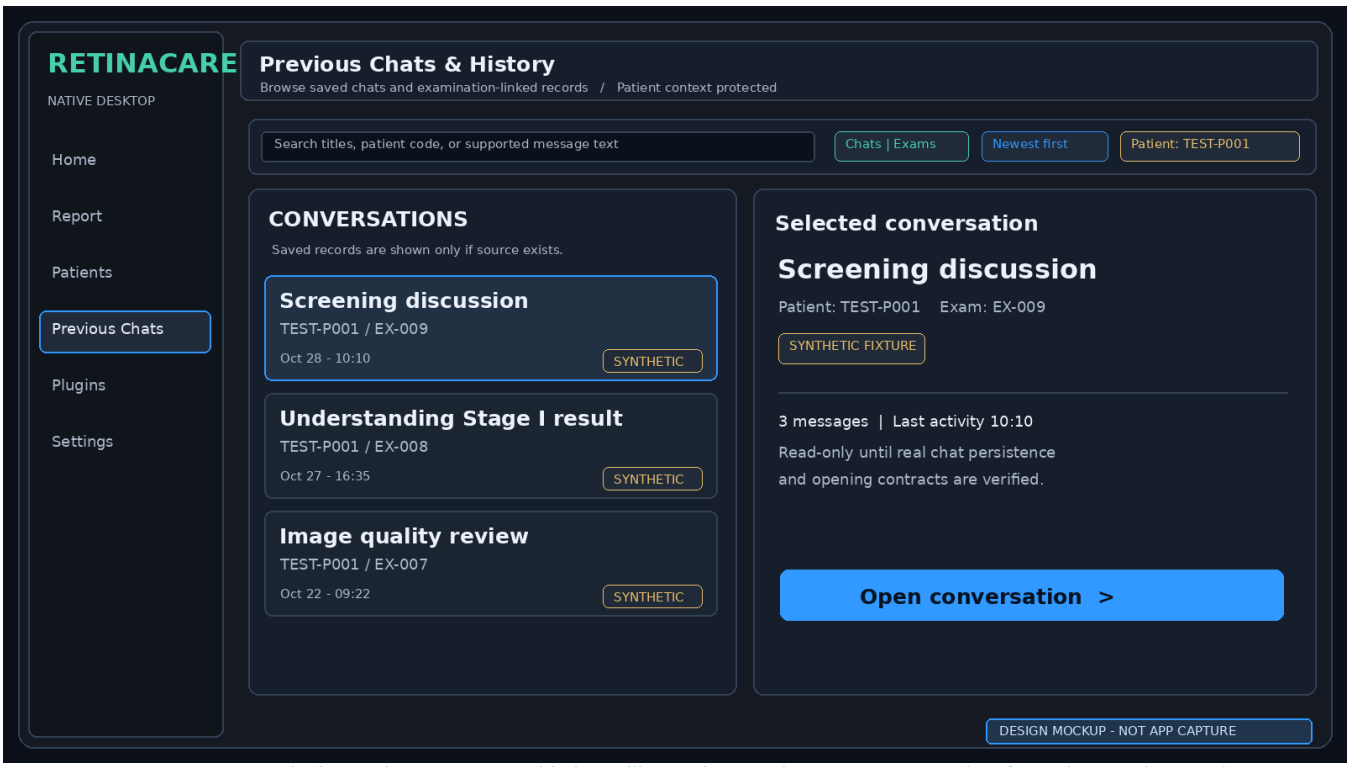
RETINACARE / ANTIGRAVITY / TASK 07

# Previous Chats & History Management

Complete native Rust GUI implementation guide: source audit, information architecture, screen designs, data contracts, privacy tests, rollout and full prompts.




DESIGN CONCEPT - desktop History screen. This is an illustrative mockup, not a screenshot from the running product.

**Current reported baseline:** Task 06.1 GUI navigation pushed as commit 82db168, 109 tests passing. A later idle responsiveness update reported 110 tests and an OpenGL Glow renderer; its commit and prolonged-idle verification are unknown and must be checked first.

**Scope:** Antigravity frontend only. Claude maintains backend, AI models, persistence, PDF and audit. Missing history services must be shown as unavailable, not faked.

**Deliverables:** This PDF contains the entire normal master prompt and a JSON master prompt in appendices; separate copy-ready .txt and .json files are supplied.

# 01 / Executive brief & baseline

The user wants a real, dedicated Previous Chats & History screen - not a row of controls stuffed into Report. Task 06.1 has already delivered routed Patient Directory, Patient Profile, Exam Detail and Report subroutes; Task 07 must extend that navigation instead of rebuilding the app. The screenshot designs later in this document are conceptual targets, not application screenshots.

| Confirmed from Antigravity report | Must verify before implementation |
|---|---|
| Task 06.1: routed patient/exam/report GUI, responsive 1280x820 and 700x650 | Current real source after latest idle/renderer changes |
| Reported commit 82db168 pushed to origin/dezine | Is the idle/Glow fix committed, pending, or overwritten? |
| 109 tests passed Task 06.1; 110 passed after idle fix | Rerun tests, exact test counts and ignored test inventory |
| Existing Previous Chats sidebar entry | Whether full conversations, histories or messages are durably persisted |
| Synthetic patient fixtures supported via RETINACARE_DEV_FIXTURES=1 | Whether real backend patient/exam/history data source is connected |

### Principle: a real UX can be finished while history storage remains blocked

GUI implementation and durable history integration are separate gates. If an existing backend history source is absent, Antigravity can complete typed routes, view states, search over demo fixtures, accessibility and truthful unavailable UI - but cannot call durable saved-chat functionality complete. Claude must implement any missing persistence APIs separately.

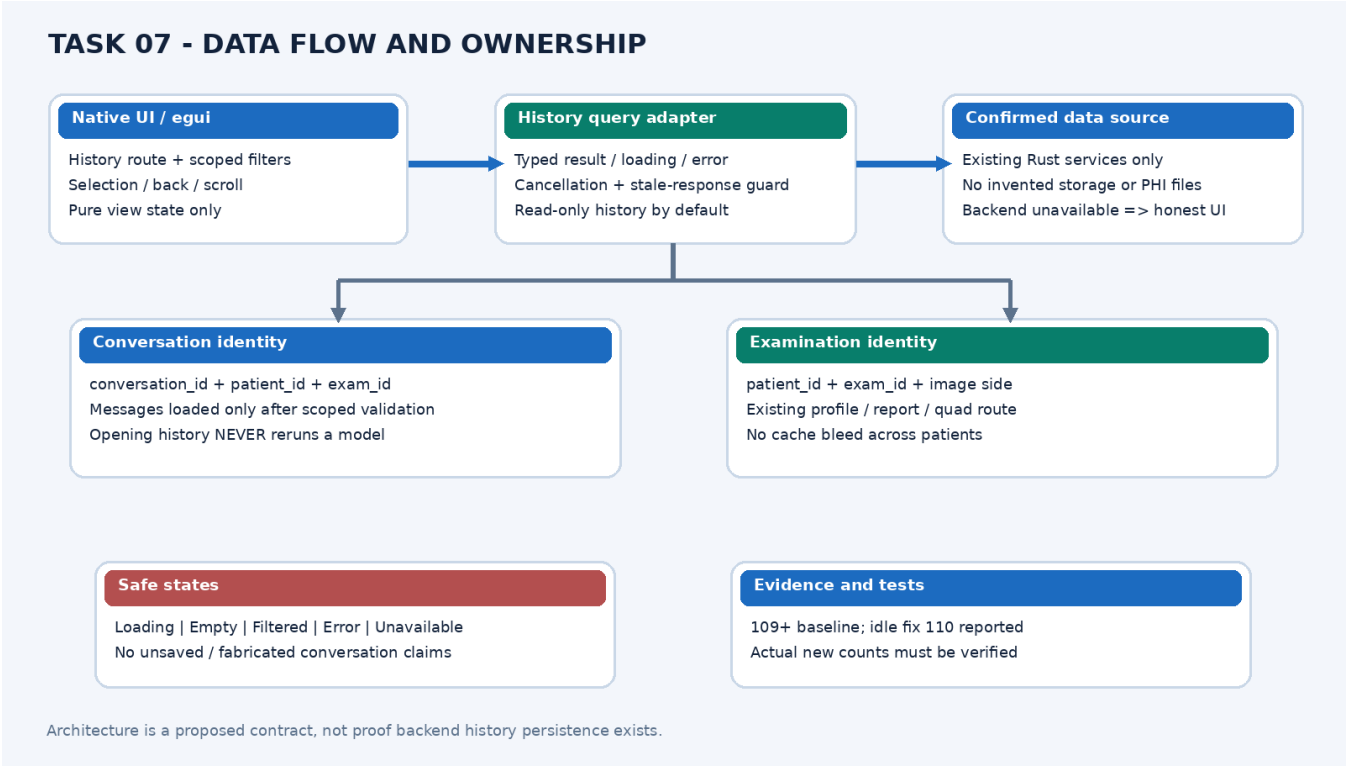
# 02 / What Task 07 includes - and excludes

| Included in Task 07 | Excluded / separate ownership |
|---|---|
| Dedicated History workspace with safe Chats and Examinations views | No new backend SQLCipher schema or filesystem PHI cache |
| Conversation selection, read-only detail, scoped search/filter/sort | No new LLM/chat inference or automatic chat replay |
| Existing examination deep links to patient/exam/report/quad | No grading or lesion-model changes |
| Real empty/loading/error/unavailable states; developer fixture labels | No invented clinical records, fake scores or mock saves |
| Responsive/keyboard UX, native screenshots, regression tests | No Task 08 Plugins work; no merging or unauthorized push |

# 03 / Proposed route and navigation architecture

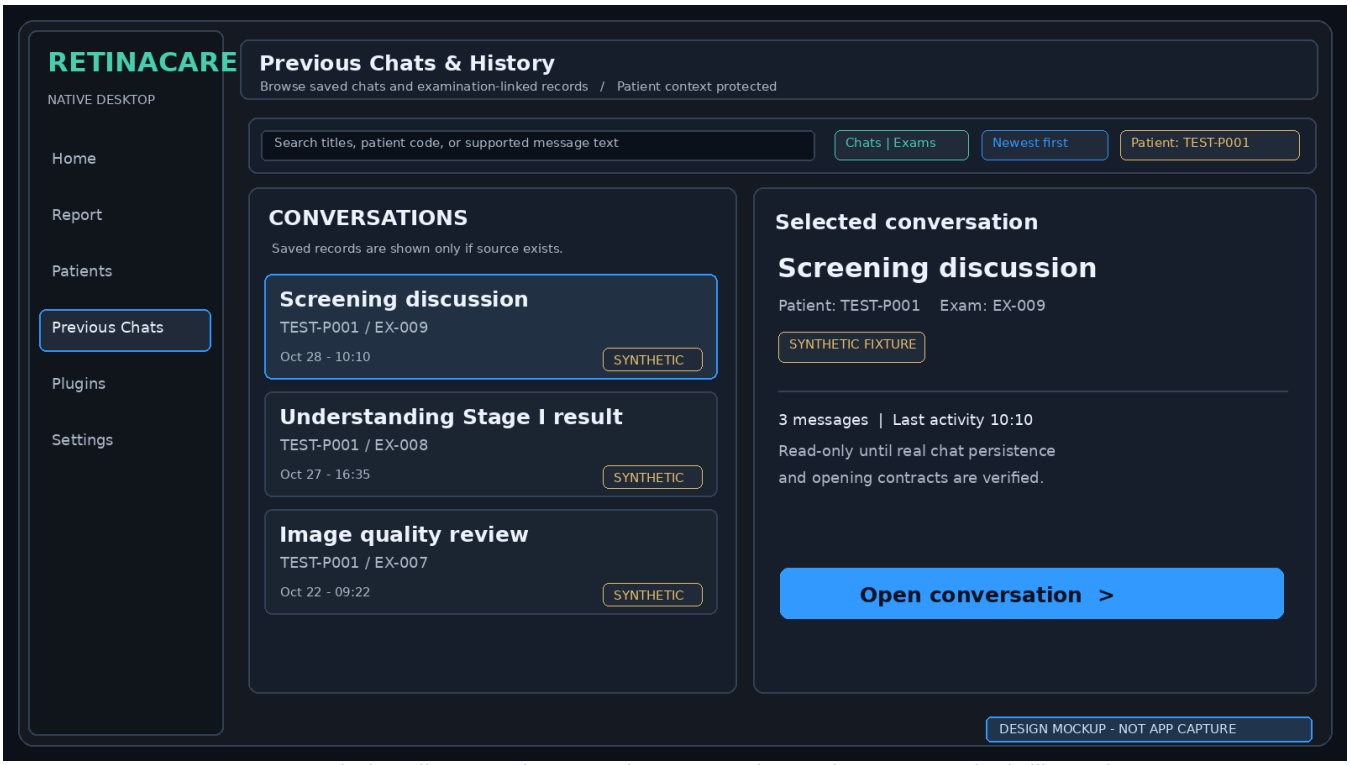
Use existing AppPage or route definitions; names below are suggestions, not mandates. Preserve the original sidebar entry. Nested History routes share one desktop shell and keep patient/exam identity explicit.

**Previous Chats** (existing sidebar) > **History Root** (Chats / Exams) > **Conversation Detail**. Examination entries deep-link into the existing Patient Profile / Exam Detail / Report Overview / Quad routes; Back should return to the original History tab and selection.




Proposed separation of view state, typed history query adapter and real existing data sources. No new backend implementation is authorized.

# 04 / Screen A - Desktop history workspace

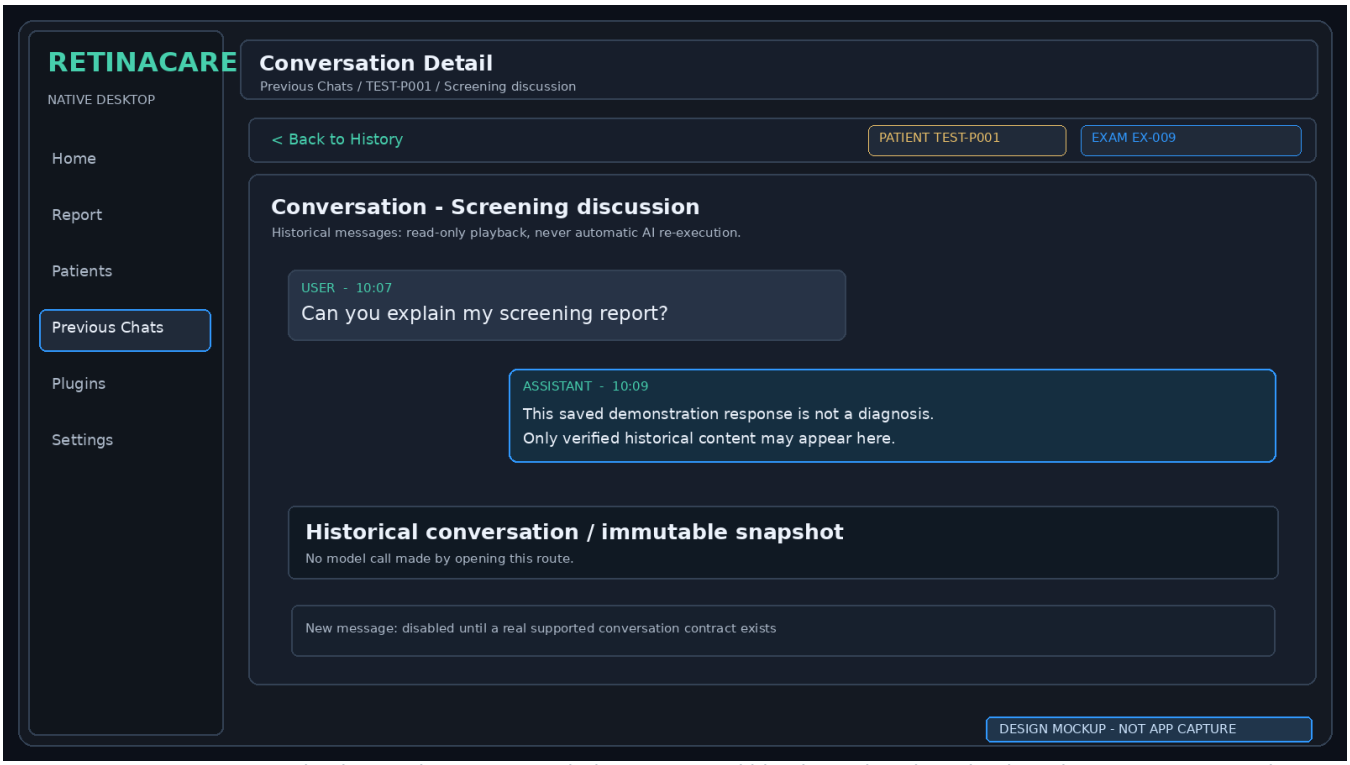



DESIGN CONCEPT - desktop list + preview. Sample names and rows shown are synthetic illustrative UI text.

## Layout requirements

- **Persistent left sidebar:** keep the established Home, Report, Patients, Previous Chats and Plugins navigation. No separate OS window and no Report-toolbar dumping ground.
- **Search and scope row:** search titles/IDs only when provided by the real source; patient/exam scoping and stable sort. A patient badge should reflect an actual scope, never an inferred patient.
- **Scrollable list:** conversation title, original activity timestamp, safe snippet, optional patient/exam link and source/fixture status. Do not display fake unread markers or invented records.
- **Context panel:** selected item metadata, safe preview and Open Conversation. Graceful empty state when no item is selected.
- **Large data:** avoid per-frame I/O and expensive unbounded searches; preserve selection and avoid row jitter or drop-down resets.

# 05 / Screen B - Conversation detail

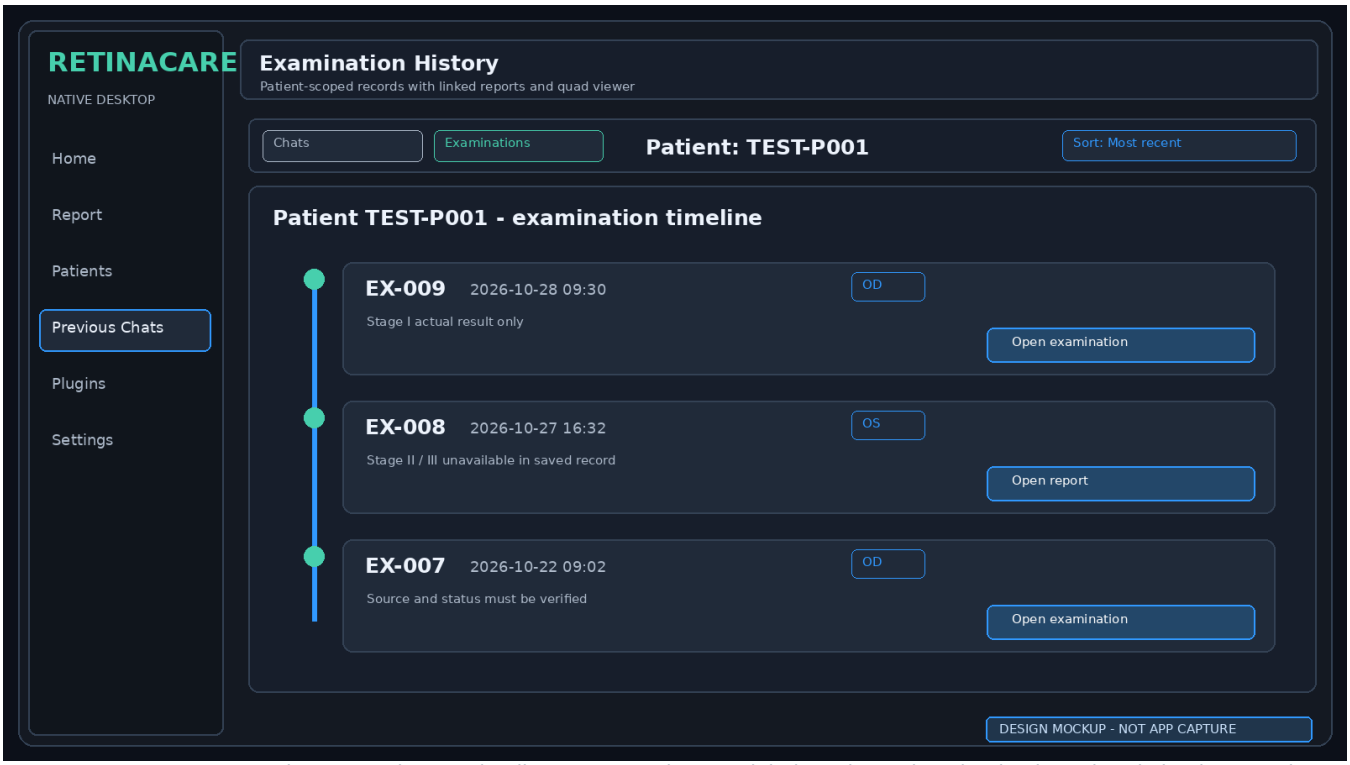



DESIGN CONCEPT - read-only saved-message rendering, contextual identity and Back navigation. Fixture message text is illustrative only.

## Behavior and correctness

Load messages by stable conversation ID and validated patient scope. Opening a historic conversation must never execute a new LLM request or medical inference. If persistent message retrieval or a genuine resume API does not exist, the composer remains disabled and the user sees an explicit reason. Preserve chronology, timestamps, safe wrapping and the exact historical body when verified from storage.

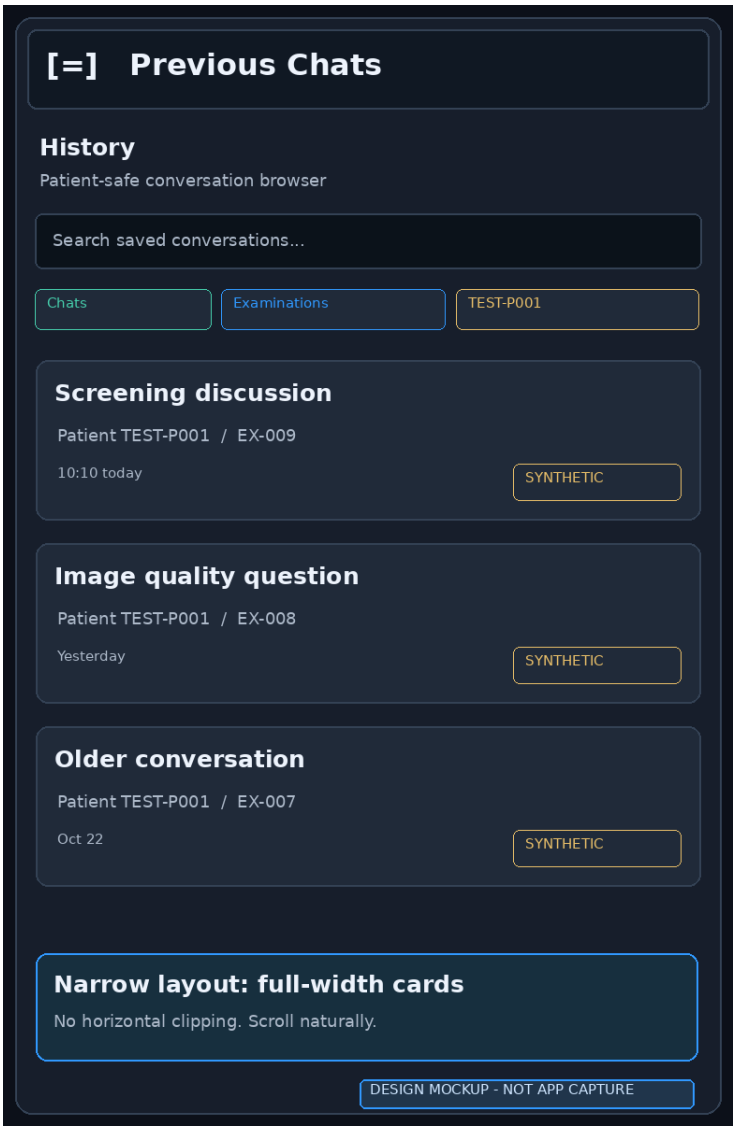
# 06 / Screen C - Examination history




DESIGN CONCEPT - patient-scoped exam timeline, contextual OD/OS labels and typed navigation into already implemented Exam/Report/Quad views.

Examination history is NOT a second database. Read from existing exam contracts only. Patient ID and exam ID must be validated together; eye laterality must not silently change examination scope. If a report is missing, display Not Available rather than Grade 0, Stage II/III prediction, or any synthetic clinical claim.

# 07 / Screen D - Narrow window




DESIGN CONCEPT - approximately 700px-wide responsive layout. Real screenshots must be captured from the compiled GUI.

At widths around 700px, use a single scrollable column with full-width cards, a compact header and no dual-column squeeze. Back/breadcrumb returns to the same history search and selection. At approximately 900px the layout may transition, but it must be tested around the breakpoint, not only at its endpoints.

# 08 / State machine and navigation contracts

| State | What user sees / key test |
|---|---|
| Loading | Progress indicator; source request pending; no patient A data in B context |
| Ready | Real records with stable sort and selection |
| Empty | Verified source returned zero records |
| FilteredEmpty | Source has records; current query returns none |
| Unavailable | No actual API / permission; never claim zero history |
| Error | Specific error, safe retry when supported |
| DemoFixture | Visibly SYNTHETIC labels in intentionally enabled DEV mode |
| StaleSelection | Record removed or mismatched patient; no accidental content bleed |

# 09 / Rust data adapters - example contracts

These are illustrative type concepts; use the actual repo types discovered in M0. Do not duplicate domain entities or establish a new backend storage system merely because an example type appears here.

```
struct HistoryQuery {
    patient_id: Option<PatientId>,
    search: String,
    sort: HistorySort,
    mode: HistoryMode,
}
enum HistoryLoadState<T> {
    Loading,
    Ready(T),
    Empty,
    Unavailable(String),
    Error { message: String, retryable: bool },
}
// Validate patient/exam binding before rendering,
// routing or accepting asynchronous query results.
```

## Proposed service boundaries

- **HistoryQuery:** patient-scope, search and sort preferences; query generation ID for stale-response isolation.
- **HistoryItemSummary:** stable ID, supported title/snippet, activity time and optional patient/exam linkage.
- **HistoryDetail:** messages in canonical saved order; do not generate content on read.
- **HistoryAdapter:** wrap actual existing service OR expose an explicit Unavailable implementation until Claude supplies persistence contract.
- **Existing exam route adapter:** only construct deep links after patient/exam identity checks; keep original source of truth.

# 10 / Privacy and reliability invariants

**Patient isolation:** when scope changes, cancel/ignore outstanding requests from the previous scope and clear sensitive cached content. **Historical view side effects:** opening old messages must not start AI inference or create a new conversation. **Security:** never write raw clinical messages, patient IDs or model responses into an unencrypted GUI-only cache, logs or screenshots in production. **Fixtures:** gated by dev mode and explicitly labeled. **Idle behavior:** preserve the reported heartbeat/Glow changes but manually test real idle/resume instead of assuming the change guarantees compositor recovery.

# 11 / Milestone-based build plan

| Milestone | Implementation target / exit evidence |
|---|---|
| M0 | Audit Git, docs, existing actual history sources; write 01-audit-and-plan.md BEFORE editing UI |
| M1 | Routed History page + desktop/narrow layouts; preserve previous sidebar and report routing |
| M2 | Typed history adapter + query/filter/sort and truthful loading/empty/unavailable/error |
| M3 | Conversation detail and read-only saved message behavior; no rerun on opening |
| M4 | Patient-scoped examinations and typed deep links to Profile/Exam/Report/Quad |
| M5 | Isolation, responsive, keyboard and idle-resume regression coverage |
| M6 | Compile/build/test, real screenshots, docs + Obsidian evidence and Task 08 handoff |

# 12 / Backend interface contract - document, do not implement

If history is not persisted or exposed in the current Rust backend, document a minimal API for Claude rather than implementing it in Antigravity. Minimum data needed: query list (patient scope, pagination/cursor, updated timestamp, stable ID, safe title), fetch detail (stored messages with roles/timestamps), safe read-only exam association, typed missing/deleted/denied/error responses and only later optional append/resume operations. Do not assume server APIs; this is a native Rust application, so prefer existing typed crate calls/channels. Any permission/access controls must be enforced at a trusted storage boundary, not only GUI filtering.

# 13 / Required tests - detailed matrix

| Area | Acceptance evidence |
|---|---|
| Routing | Sidebar to History, open detail, return/back and preserve filters; no Report/Quad regressions |
| Search/filter | Stable results; supported fields only; no-match vs truly empty distinction |
| Patient switch | In-flight result from A discarded if B selected; no A messages or images on B |
| Message history | Stored ordering/timestamps; no new inference when reopening |
| Exam links | Known patient/exam pair routes to correct existing Profile/Exam/Report/Quad |
| Data states | Loading, Empty, FilteredEmpty, Error, Unavailable, DEMO badge all truthful |
| Safety | No GUI plaintext PHI cache; no phantom saved chats; deletion/missing IDs safe |
| Responsiveness | 1280x820, 1024x768, ~900px and 700x650; long titles and many rows |
| Keyboard | Focus, Enter, Escape/Back, scrolling; behavior documented |
| Idle regression | Reported idle fix remains; 20+ minute real idle and monitor resume checked if possible |
| Regression | Task01-06.1 and native Stage I integration remain unaffected |

## Commands - execute, do not infer results

```
cargo fmt --manifest-path rust/Cargo.toml --all -- --check
cargo check --manifest-path rust/Cargo.toml
cargo clippy --workspace --all-targets --manifest-path rust/Cargo.toml -- -D warnings
cargo test --workspace --manifest-path rust/Cargo.toml
cargo build --release --manifest-path rust/Cargo.toml
git diff --check
```

The report must state the actual current Git HEAD, pass/fail/ignored counts and screenshot environment. A unit test for a repaint interval does not by itself verify compositor recovery after sleep.

## 14 / Screenshot and documentation contract

Capture genuine screenshots from the running native compiled application, not AI-generated mockups, and mark `RETINACARE_DEV_FIXTURES=1` when used. Recommended screenshot names are listed in the JSON appendix. Include actual source file changes, test logs, backend blockers and a truthful GUI-complete vs persistence-blocked verdict. Update the GUI Obsidian vault and verify all wikilinks; do not modify Claude backend memory.

## 15 / Definition of done

- History is a real native routed page with usable desktop and narrow layouts.
- Legitimate list/query/detail states are implemented; unavailable is not confused with empty.
- Conversation viewing is read-only by default, with no accidental AI rerun or cross-patient leak.
- Examination timeline and typed deep links reuse the actual existing source of truth.
- Tests, release build and actual native screenshots are recorded.
- Docs, Obsidian handoff and exact changed-file report exist and remain honest.
- If no persistent chat service exists, report GUI complete / persistence integration partial, not full Task 07 service completion.
- No backend changes, no unapproved commit/push, no Task 08 startup.

# APPENDIX A / Complete normal master prompt for Antigravity

Copy the companion `.txt` file or use the full prompt reproduced below. This appendix is complete, not an abbreviated excerpt.

### RETINACARE | ANTIGRAVITY | TASK 07

### PREVIOUS CHATS & HISTORY MANAGEMENT - NATIVE RUST GUI

### TASK OWNER: ANTIGRAVITY (FRONTEND ONLY)

## MISSION

Implement a polished and functional Previous Chats & History experience in the existing RetinaCare native egui/eframe desktop app. Task 06.1 established dedicated routed patient/profile/exam/report screens and pushed GUI commit 82db168 on origin/dezine. The subsequent idle-UI heartbeat/Glow renderer fix was reported with 110 tests, but its Git status and prolonged-idle behavior must be verified. Task 07 must build on that actual state rather than replace it. Create history browsing, safe conversation opening, examination-linked navigation, search/filter/sort, resilient loading/empty/error/unavailable states, desktop/narrow responsive layouts, and meaningful privacy tests. Do not fabricate backend persistence or chat/clinical content.

## IMPORTANT FIRST STEP - READ MEMORY, INSPECT SOURCE, WRITE THE PLAN

1. Verify actual GUI repo, Git branch/HEAD, clean/dirty working tree and the status of the idle/Glow fix. The anticipated GUI root is /home/claude/Desktop/Hackthon dezine/rust and the owner vault is /home/claude/Desktop/Hackthon dezine/memory. Don't assume source filenames, API availability, or that the fix was committed.

2. Read AGENTS.md, the GUI vault README/start-here, active-phase, GUI Task 06.1 handoff docs (docs/features/GUI-P06-patient-navigation-redesign/), relevant Previous Chats/chat/task02 notes, and relevant Ponytail/Superpowers skills already installed. Avoid scanning the entire repository or blindly rereading the entire vault.

3. Source-audit existing AppPage navigation, patient/exam subroutes, ChatService and message data models, any conversation identifiers, history persistence interfaces, active patient context, fixture mode, query mechanics, and current tests. Record EXACTLY what is real, mocked, absent or disconnected.

4. BEFORE coding, create docs/features/GUI-P07-previous-chats-history/01-audit-and-plan.md containing screen/route map, source references, data-contract risks, state machine, file-by-file scoped plan, invariants, test matrix and planned screenshots.

5. Only then implement Task 07. Do not silently convert a missing backend history feature into a GUI-owned database.

## OWNERSHIP / ABSOLUTE LIMITS

- Native Rust GUI only: egui/eframe; use current repository architecture. No HTML/CSS/JS/TS/React/Next.js/Electron/Tauri/WebView or browser-hosted components.

- Claude exclusively owns Rust backend domain/persistence/model engine, PDF and audit implementations and AI stage work. Do not edit /home/claude/Desktop/Dird or the backend Obsidian vault /home/claude/Desktop/Hackthon dezine/Hackthon Backend. Do not merge repositories, modify model weights or invent HTTP endpoints.

- Preserve Task 01-06.1 behavior, especially report/quad controls, patient directory/profile/exam details, typed navigation and genuine Stage I integration; preserve the idle fix unless source evidence warrants a narrowly justified correction.

- Do not alter existing branch history, commit, push or create PR without direct user authorization. Leave unrelated worktree modifications untouched.

- Every medical or conversation fixture must remain clearly marked SYNTHETIC/DEMO; no invented patient histories, no displayed fabricated grading or medical diagnoses; no shadow local plaintext patient/chat persistence.

- For inaccessible data or missing services, visibly display Backend Unavailable or History Not Connected, not an empty list falsely implying no records.

## SCOPE A - PREVIOUS CHATS ROOT SCREEN

- The existing sidebar Previous Chats entry must navigate to a dedicated full-content routed History screen within the same OS window (not a modal or a toolbar tab in Report). Keep sidebar and responsive page structure from Task 06.1.

- Desktop layout (~1280x820 and larger): search and scope controls above a scrollable conversation list with a selected conversation preview/detail area. A master-detail split or a strongly justified alternative is acceptable; don't leave a huge unused void or cramped cards.

- Narrow layout (~700x650 and smaller): single-column list, full-width controls, clean navigation to a full-page detail screen, no clipping, focus loss or horizontal overflow.

- Meaningful header, record count only if the source can supply it, sorted recent activity, timestamps in a clearly indicated timezone, source/readiness badge, patient/exam link when truly present, and unread markers ONLY if real source supports them.

- Conversational item contains stable conversation ID, safe title/snippet, last activity, known patient/exam IDs, source state and navigational affordance. Never synthesize a conversation preview from a predicted diagnosis or system prompt.

- Design must support lots of records via scrolling/virtualization if needed; avoid O(N^2) string search every egui frame and avoid heavy I/O in render calls.

## SCOPE B - SEARCH, FILTER, SORT, EMPTY AND ERROR STATES

- Search by safe available fields such as title, conversation code and patient code. Message-body search only when backed by a legitimate source and scoped/authorized; do not claim indexed body search without implementing it.

- Filters: conversation vs examination (if those data sets exist), active patient scope, optional date range or eye side ONLY if supported by fields; no fabricated fake counts.

- Sort most recently active by default; stable tie-breaker (ID), sensible ascending option, deterministic search results, preserve selection on harmless filter changes, gracefully clear invalid selection.

- Distinguish LOADING, EMPTY (zero source records), NO MATCHES (filter produces zero), UNAVAILABLE (service not implemented), ERROR (query failed), STALE/DELETED and DEMO FIXTURE modes. User gets meaningful Retry only when retry can really execute.

- Search query, list selection and filter state survive opening a detail view and pressing Back without leaking across patient scopes.

## SCOPE C - CONVERSATION DETAIL / REOPEN SAFELY

- Separate subroute for an existing conversation with breadcrumb/back, title, original message timeline, timestamps, stable roles, and patient/exam context when linked.

- Reopening a saved conversation must LOAD existing messages; never regenerate/re-run an LLM, Stage I/II/III inference, report export or audit event merely by opening history.

- If there is no durable conversation store or no verified full history API, ship the polished GUI with an explicit read-only/unavailable state and typed adapter contract; avoid pretend 'saved' messages. In DEV fixture mode, synthetic test messages must be unmistakably labeled.

- If the current project really supports resuming a conversation, preserve its known conversation ID and patient/exam scope; disable composing when resume/write services are unavailable. Do not accidentally attach a new message to the wrong patient or create a new conversation behind the user's back.

- Rendering history must not duplicate long messages or expose hidden system prompts, medical secrets, raw access tokens or unvalidated markdown/links. Use existing safe message display components and graceful wrapping for long text.

## SCOPE D - EXAMINATION HISTORY AND CROSS-ROUTE DEEP LINKS

- Provide an Examination History view within the History workspace only if an actual exam source exists. Reuse existing patient/exam domain models and records; do not duplicate the clinical database in GUI state.

- Patient-specific chronological timeline, known exam ID, capture date, laterality OD/OS, genuine readiness/status. A missing grade is 'not available', never a fabricated Grade 0.

- Open existing Patient Profile, Exam Detail, Report Overview or Quad through typed navigation with patient_id + exam_id + eye. Validate record belongs to current patient before routing and clear any stale cached image/report on scope switch.

- Back / breadcrumbs preserve originating History tab, scope, filter, selection and scroll position as feasible. No hidden reruns or saves when navigating.

- If exam history is present but chat history is not, show the former honestly and the latter as unavailable. If both absent, show accurate service status instead of filler data.

## SCOPE E - TYPED HISTORY SERVICE ADAPTER (GUI-OWNED BRIDGE ONLY)

- Reuse existing ChatService or typed client API when it truly exists. Define minimal GUI-level HistoryQuery, HistoryItemSummary, HistoryDetail, HistoryLoadState, HistoryError and HistoryRoute IDs as appropriate. Audit current types FIRST and extend instead of creating duplicate conflicting enums.

- A GUI adapter may map existing persisted backend records or fixture input into view models. If Claude has not provided a history persistence API, record requested backend contracts in documentation and use an explicit Unavailable implementation, not new SQLCipher tables, local JSON storage, or a new backend crate.

- Keep all I/O, sorting expensive data, and blocking fetches off the egui render loop. Use current async/thread/channel setup, cancellation/epoch guards, bounded memory, and egui repaint requests only when state changes. Preserve reported 500ms idle heartbeat and Glow renderer configuration; verify rather than treat them as proven sleep-resume fixes.

- Stale response and scope-change isolation: a response for patient A must never appear after patient B is activated. Ignore old generation IDs, clear sensitive cached details and enforce matching IDs before displaying.

## UI DESIGN & VISUALS

- Maintain current premium charcoal/native UI: consistent spacing, subtle blue/teal selection, status labels, accessible typography, keyboard focus, discoverable breadcrumbs and contextual controls.

- Design examples in the attached PDF are CONCEPT MOCKUPS, not screenshots of implementation; visual fidelity is judged on an actually running native app.

- No 'Previous Chats' content buried in Report toolbar. No second OS window. Avoid dashboard decoration that misrepresents data. Keep the sidebar hierarchy and existing app modes unchanged.

- Keyboard: focus order for search/list/details, Enter to open, Escape/back where consistent, Up/Down list navigation where supported; mouse/scroll; screen reader limitations documented honestly if platform API does not expose accessibility.

- Responsiveness: verify 1280x820, 1024x768, 900px boundary, 700x650, small dialog and long text/language cases. Avoid pixel-perfect hardcoding when using egui layout.

DESIRED STATE MACHINE (illustrative)

HistoryState::Loading, Ready {records, selected_id}, Empty, FilteredEmpty, Unavailable {reason}, Error {retryable}, Detail {id, messages, source_state}; typed patient/exam scope separate from route. Choose real names based on code. On filter/scope change validate selection and cancel stale data. Never render 'no history' on unavailable service.

## IMPLEMENTATION MILESTONES

### M0. Audit and plan: document existing history sources, route map, backend gaps and baseline Git state. No unsupported storage promises.

### M1. Add Previous Chats primary History page, stable typed routes, desktop master-detail and narrow single-column view; preserve sidebar navigation.

### M2. Add legitimate data adapter and deterministic query/filter/sort handling with all distinct load/empty/error/unavailable states.

### M3. Build conversation detail/read-only playback and optional genuine safe resume only if available; verify no rerun on open.

### M4. Build patient-scoped examination history with links to existing Profile/Exam Detail/Report/Quad; reuse existing data contracts and never show invented scores.

### M5. Complete privacy/stale-response protections, keyboard interactions, responsive polish, and prolonged-idle functional smoke tests.

### M6. Run all gates, launch native binary, exercise real desktop and narrow routes, capture verified screenshots, publish docs and Obsidian handoff. DO NOT START TASK 08 automatically.

## REQUIRED TESTS AND EVIDENCE

- Sidebar Previous Chats -> History and back to prior main route works; no report/quad regression.

- Scoped search, filters and stable sorting; result count correctness; no matches vs no records vs unavailable vs error.

- Historical detail preserves original ordering, does not trigger model/LLM calls, and exposes no raw system prompts.

- History -> Patient Profile / Exam Detail / Report / Quad, then back, preserving IDs and selection.

- Switching patient A to B while a background fetch for A is pending shows no A messages, exams or images in B view.

- Malformed/deleted IDs, missing patient link, empty chat, long chat, many records, long Unicode title, mixed OD/OS, single exam, no exam, disabled service.

- Synthetic fixtures are explicitly labeled and only enabled through recognized dev flag; no fixtures masquerading as clinical history.

- Read-only mode: no fake save, fake resumption or phantom file on disk; error and Retry only for actual request capability.

- Keyboard Tab/Enter/Escape, mouse, scroll, narrow widths, no clipping/overlap; 20+ minute idle and real monitor sleep/resume clicks tested or clearly marked unverified (do not claim a 500ms heartbeat guarantees Wayland recovery).

- Existing Task 01-06.1 tests + user-reported idle fix regressions, all relevant Rust tests.

RUN THESE GATES FROM ACTUAL GUI REPO ROOT (confirm paths first)

cargo fmt --manifest-path rust/Cargo.toml --all -- --check

cargo check --manifest-path rust/Cargo.toml

cargo clippy --workspace --all-targets --manifest-path rust/Cargo.toml -- -D warnings

cargo test --workspace --manifest-path rust/Cargo.toml

cargo build --release --manifest-path rust/Cargo.toml

git diff --check

If already inside rust/, omit the rust/ prefix; do not claim a gate passed unless actually run. Record exact counts, ignored tests, warnings, build target, and the current HEAD. Use existing repo scripts when they are authoritative.

## REAL SCREENSHOT / VISUAL VERIFICATION

Launch the actual compiled native app in Ubuntu Wayland/X11 as supported; use RETINACARE_DEV_FIXTURES=1 only to demonstrate explicitly synthetic records. Capture actual screenshot evidence in docs/features/GUI-P07-previous-chats-history/: history_desktop_verified.png, history_detail_verified.png, examination_history_verified.png, history_empty_unavailable_verified.png, history_narrow_verified.png, patient_scope_isolation_verified.png (only when reproducibly verified), idle_resume_verified.png (only when manually tested). Label screenshot environment and fixture mode in captions. PDF reference mockups are NOT verification artifacts.

## DOCUMENTATION AND OBSIDIAN

Write a scoped documentation suite in docs/features/GUI-P07-previous-chats-history/:

01-audit-and-plan.md

02-route-and-state-design.md

03-history-data-contracts.md

04-implementation-and-file-changes.md

05-tests-and-security-evidence.md

06-screenshot-and-responsive-review.md

07-blockers-and-claude-backend-contracts.md

08-task08-handoff.md

Update only Antigravity-owned GUI canonical vault /home/claude/Desktop/Hackthon dezine/memory: active-phase, current-state, routes/history, decisions, gotchas, testing, changelog and handoff as relevant. Preserve existing folder layout and the <=80-lines-per-note guideline from Task 06.1; use working [[wikilinks]], verify targets, avoid touching Claude's backend vault. Memory notes may be stale; match source evidence. Store future backend requests as interface contracts only, no edits to backend code.

## FINAL REPORT FORMAT

1. Verdict: COMPLETE / PARTIAL / BLOCKED, separately for GUI behavior and durable history integration.

2. Git branch/HEAD, working tree before/after, idle fix status, and actual source files changed.

3. Feature matrix: Chats, search/filter/sort, detail, examinations, routing, persistence readiness, error/empty states, accessibility/responsive.

4. Data authenticity: real persistence vs synthetic fixtures vs backend unavailable; precise blockers with API contract required of Claude.

5. Tests: exact commands, passed/failed/ignored numbers, and screenshot evidence of the compiled running app.

6. Patient identity isolation, storage security and no-reexecution proof.

7. Obsidian vault notes updated, links verified, and next Task 08 handoff.

8. Explicitly state NO BACKEND OR GUI MERGE, NO COMMIT/PUSH unless separately approved.

## 100% TASK 07 ACCEPTANCE CHECKLIST

[ ] Existing Previous Chats sidebar route loads a real dedicated screen in the same native app.

[ ] History list and desktop/narrow layouts work without clipping; no huge unused dead space.

[ ] Correct distinction among real history, synthetic dev fixtures and unavailable backend.

[ ] Search, patient scope, stable sort, filters, empty/no-match/unavailable/error and Retry states behave truthfully.

[ ] Selected conversation detail opens, scrolls, returns to list and never silently reexecutes inference.

[ ] Conversation identifiers, patient_id and exam_id are preserved and validated.

[ ] Examination history reuses actual existing records and correctly links to patient/exam/report/quad.

[ ] Cross-patient isolation and stale response tests pass; no cross-exam image/result bleed.

[ ] Disabled functionality is clearly described; no fake saved messages, database or exports.

[ ] Keyboard and responsive interactions tested at multiple sizes; long content safe.

[ ] Idle fix regression verified, with manual idle/sleep outcomes clearly distinguished from unit tests.

[ ] Existing UI and Stage I integrations unaffected; full fmt, check, strict clippy, tests, release build and diff check pass.

[ ] Native screenshots captured only from actual running app and marked if synthetic fixture mode.

[ ] File-by-file documentation, API blockers and GUI Obsidian memory updated with working links.

[ ] Correct honest verdict: if durable conversation persistence is missing, report GUI COMPLETE but history-persistence integration PARTIAL/BLOCKED.

[ ] Do not begin Task 08, change Claude backend, commit or push without authorization.

START TASK 07 NOW: Plan briefly first, then implement real native Rust GUI code and verify it. Do not stop at mockups or a purely conceptual report. Keep the task scoped to previous chats/history.

# APPENDIX B / Complete structured JSON master prompt

This appendix reproduces the full structured prompt supplied as a standalone JSON file. JSON keys are descriptive integration instructions, not executable code.


```
{
  "project": "RetinaCare",
  "agent": "Antigravity",
  "task_id": "GUI-T07",
  "name": "Previous Chats & History Management",
  "version": "1.0",
  "language": "Rust",
  "ui_framework": [
    "egui",
    "eframe"
  ],
  "frontend_only": true,
  "context": {
    "last_known_gui_commit": "82db168 (origin/dezine, reported Task06.1)",
    "last_known_tests": "109 at Task06.1; 110 after claimed idle fix - VERIFY",
    "repo_expected": "/home/claude/Desktop/Hackthon dezine/rust",
    "gui_vault": "/home/claude/Desktop/Hackthon dezine/memory",
    "backend_repo_do_not_edit": "/home/claude/Desktop/Dird",
    "backend_vault_do_not_edit": "/home/claude/Desktop/Hackthon dezine/Hackthon Backend",
    "backend_owner": "Claude",
    "no_commit_or_push_without_approval": true
  },
  "mission": "Ship a trustworthy responsive native Previous Chats/History UX, grounded in
actual saved data if available; otherwise explicit backend-unavailable states and typed
integration contracts, without inventing persistence or rerunning inference.",
  "first_actions": [
    "Verify git and uncommitted idle fix; keep user changes",
    "Read AGENTS.md, relevant installed skills, GUI Obsidian start/active phase and Task06.1
 docs",
    "Audit code for real chat/exam history sources, routes and storage; inspect only
relevant modules",
    "Write M0 audit and implementation plan before coding"
  ],
  "hard_constraints": [
    "Native Rust egui/eframe only; no web technology",
    "No edits to Claude backend, weights or backend vault",
    "No new GUI-owned clinical data store, SQL tables or PHI plaintext files",
    "No fake conversations or unlabelled demo data",
    "No model/LLM inference triggered by opening history",
    "No cross-patient leaks, wrong-exam report or image selection",
    "No rewriting Task01-06.1, no Task08 implementation",
    "No commit, push, or merge without user authorization"
  ],
  "routes": [
    {
      "name": "HistoryRoot",
      "location": "existing Previous Chats sidebar",
      "behavior": "dedicated screen with Chats/Exams modes and scope"
    },
    {
      "name": "HistoryConversationDetail",
      "location": "nested History screen",
      "behavior": "read-only existing conversation and verified resume if truly available"
    },
    {
      "name": "HistoryExamTimeline",
      "location": "nested History screen",
```

```
      "behavior": "patient-scoped exam list with deep links into existing typed routes"
    }
  ],
  "features": [
    {
      "id": "T07-A",
      "name": "History root and navigation",
      "must_have": [
        "desktop master-detail",
        "narrow single column",
        "persistent sidebar",
        "back/breadcrumbs",
        "real scope badges"
      ]
    },
    {
      "id": "T07-B",
      "name": "Search/filter/sort",
      "must_have": [
        "safe available fields only",
        "patient scoped",
        "stable order by activity and id",
        "no records vs no matches distinct",
        "loading/error/unavailable distinct"
      ]
    },
    {
      "id": "T07-C",
      "name": "Conversation detail",
      "must_have": [
        "load original messages without reexecuting",
        "stable IDs and timelines",
        "no invented message body",
        "composer disabled until actual supported resume"
      ]
    },
    {
      "id": "T07-D",
      "name": "Examination history",
      "must_have": [
        "reuse existing actual exam records",
        "OD/OS and date only if known",
        "deep links to Profile, ExamDetail, Report, Quad",
        "no invented DR grade"
      ]
    },
    {
      "id": "T07-E",
      "name": "Typed GUI history adapter",
      "must_have": [
        "reuse real interface where present",
        "stateful ready/loading/error/unavailable",
        "async nonblocking",
        "epoch cancellation guards",
        "no GUI DB"
      ]
    },
    {
      "id": "T07-F",
      "name": "Security and responsiveness",
      "must_have": [
        "cross-patient isolation",
```

```
      "clear old selections",
      "keyboard and scroll",
      "responsive widths 1280,1024,900,700",
      "prolonged idle regression"
    ]
  },
  {
    "id": "T07-G",
    "name": "QA, screenshots and memory",
    "must_have": [
      "native Ubuntu smoke tests",
      "fmt/check/clippy/test/release/diff",
      "real screenshots",
      "GUI Obsidian notes",
      "exact changed files and honest verdict"
    ]
  }
],
"milestones": [
  {
    "id": "M0",
    "goal": "Audit and scoped plan",
    "evidence": "01-audit-and-plan.md with actual contracts and tests"
  },
  {
    "id": "M1",
    "goal": "Dedicated routed History UI",
    "evidence": "desktop and narrow screenshot evidence"
  },
  {
    "id": "M2",
    "goal": "Source adapter and truthful query states",
    "evidence": "search/sort/loading/error/unavailable tests"
  },
  {
    "id": "M3",
    "goal": "Conversation read-only detail",
    "evidence": "no-rerun and identity tests"
  },
  {
    "id": "M4",
    "goal": "Existing exam history/deep links",
    "evidence": "patient and exam route tests"
  },
  {
    "id": "M5",
    "goal": "Security, keyboard, responsive, idle",
    "evidence": "isolation and manual testing report"
  },
  {
    "id": "M6",
    "goal": "All gates and handoff",
    "evidence": "screenshot inventory, verified tests, Obsidian links"
  }
],
"readiness_states": [
  "Loading",
  "Ready",
  "Empty",
  "FilteredEmpty",
  "BackendUnavailable",
  "Error",
```

```
    "StaleSelection",
    "DemoFixture"
  ],
  "security_invariants": [
    "conversation_id/patient_id/exam_id verified before display",
    "discard stale async results on patient or route switch",
    "historical open never reruns AI or LLM",
    "no plaintext PHI export or duplicate data store",
    "fixtures labeled SYNTHETIC and opt-in only",
    "backend unavailable not disguised as zero records"
  ],
  "qa": {
    "commands": [
      "cargo fmt --manifest-path rust/Cargo.toml --all -- --check",
      "cargo check --manifest-path rust/Cargo.toml",
      "cargo clippy --workspace --all-targets --manifest-path rust/Cargo.toml -- -D
warnings",
      "cargo test --workspace --manifest-path rust/Cargo.toml",
      "cargo build --release --manifest-path rust/Cargo.toml",
      "git diff --check"
    ],
    "command_note": "Adjust manifest path after checking cwd; run actual commands, do not
invent results",
    "screen_sizes": [
      "1280x820",
      "1024x768",
      "900px breakpoint",
      "700x650"
    ],
    "screenshots": [
      "history_desktop_verified.png",
      "history_detail_verified.png",
      "examination_history_verified.png",
      "history_empty_unavailable_verified.png",
      "history_narrow_verified.png",
      "patient_scope_isolation_verified.png if verified",
      "idle_resume_verified.png if actually manually verified"
    ],
    "test_matrix": [
      "navigation and back/breadcrumbs",
      "query/sort/filters",
      "empty vs filtered-empty vs unavailable",
      "long messages/Unicode/many records",
      "scope switch mid-fetch",
      "conversation load does not trigger inference",
      "exam deep-link identity",
      "synthetic fixture gate",
      "keyboard/narrow",
      "prolonged idle/resume manual test"
    ]
  },
  "documentation": {
    "root": "docs/features/GUI-P07-previous-chats-history/",
    "files": [
      "01-audit-and-plan.md",
      "02-route-and-state-design.md",
      "03-history-data-contracts.md",
      "04-implementation-and-file-changes.md",
      "05-tests-and-security-evidence.md",
      "06-screenshot-and-responsive-review.md",
      "07-blockers-and-claude-backend-contracts.md",
      "08-task08-handoff.md"
```

```
    ],
    "vault": "/home/claude/Desktop/Hackthon dezine/memory",
    "rules": [
      "reuse layout",
      "<=80 lines per note per reported convention",
      "resolve all [[wikilinks]]",
      "update source-grounded status only"
    ]
  },
  "acceptance": {
    "gui": "Complete only if all routed responsive screens and trustworthy states verified
by real tests",
    "durable_history": "Partial or Blocked if no actual persisted backend chat interface
exists",
    "do_not_mark": "No full persistence completion based solely on fixture tests",
    "final_report_sections": [
      "separate GUI/integration verdict",
      "git HEAD/worktree",
      "changed files",
      "feature matrix",
      "test commands and counts",
      "real screenshots",
      "privacy/no-rerun proof",
      "backend blockers",
      "vault handoff"
    ]
  },
  "complete_normal_prompt_included_in_pdf": true
}
```